

NKLCLIB API user manual

Version: 5.0.0

Date: 23/12/05

Author: EA

NKLCLIB API user manual

5.0.0

2023-12-5

Distribution list:

Name / Group	Company
EA	

Reviews/Approvals:

	Name / Function / Company	Signature
Author:	EA	
Reviewed by:		

Table of Contents:

1 History	3
2 Introduction	4
3 Installation	5
4 API usage	6
4.1 API function list	6
4.2 Light Control Library	7
4.2.1 Light Control Library Base	7
4.2.2 Light Control Functions in async mode	8
4.2.3 Light Control Functions	21
5 Test Tool	32
5.1 Light Control Test	32
5.1.1 Advanced	33
5.2 Firmware update	34
6 Development	36
7 FAQ	37
7.1 What's the serial communication configure parameters?	37
7.2 How to check the hardware version?	37
7.3 How to get the configure profile for different IO board?	37

1History

Version	Date	Author	Description
1.0.0	2020-7-10	EA	Initial version
4.0.0	2020-12-16	EA	For hardware version 4.0
4.0.1	2020-12-21	EA	Fix some bug, add the C# example
4.1.0	2021-01-28	EA	Optimize the API to be compatible with hardware version 2.x
4.1.5	2021-04-13	EA	Add the API description for the holding time unit of the light control which is only available for the firmware version from 4.2.3.
4.1.6	2021-04-18	EA	Change DIO function description: for the DO, set 0 to turn on and set 1 to turn off.
4.1.7	2021-05-12	EA	Add the APIs to read back the digital output by bit, byte and word in the polling mode. See Polling Mode .
4.1.8	2021-07-27	EA	Add the light control mode contents. See LC_SetPwmParams
4.1.9	2021-08-20	EA	Add the API to turn on/off the light without save. See LC_SetPwmParamsNoRetentive
5.0.0	2023-12-01	EA	Modify the library.Separate DIO and light control into two libraries.This library is for Light Control.

2Introduction

Nodka NP-61xx Automation PC series products provide the interface to extend the functionalities by the add-on board.

For the PWM light control features, Nodka provides the dynamic library and export the function interfaces to be called by the user's program to access the data.

The Pwm light control function will be illustrated in this document. Currently, the library and driver supports the below OS:

- Windows 7 (32-bit & 64-bit)
- Windows 8 (32-bit & 64-bit)
- Windows 10 (32-bit & 64-bit)

For the other OS to be supported, please contact Nodka support for the further information.

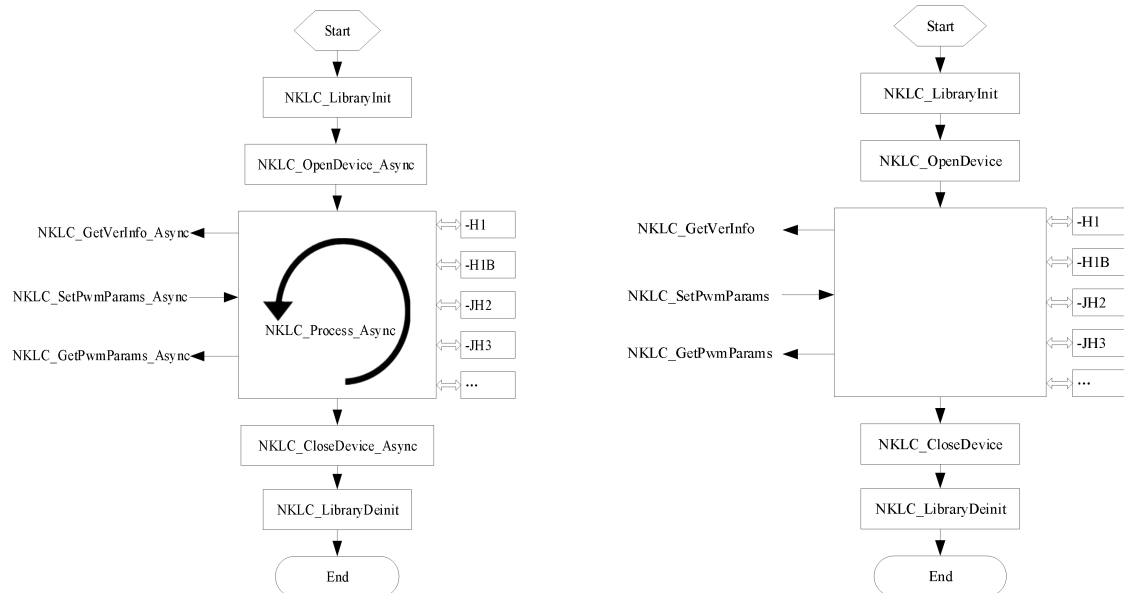
3Installation

The drivers and library files can be decompressed by installing the 'NKDIOLC_Driver_Setup_x86.exe' file. The default installation path is 'C:\NODKA', a folder named NKDIOLC_SDK will be found after the driver installed successfully. The below is the contents list in the NKDIOLC_SDK folder.

 Bin	■ Bin: Test tool working folder, which can be used to test the functionality without coding.
 ConfigFile	■ ConfigFile: Configuration files for different models.
 Include	■ Include: the header files including the API declaration, which can be included in the C/C++ development.
 Lib	■ Lib: library binary files which will be linked to the execute files.
 Manual	■ Manual: user manual for the SDK.
 Sample	■ Sample: some examples to be referenced during the development.
 vc_redist	■ vc_redist: VC++ runtime library, you must install them manually if your PC has not installed.
 History.txt	

4API usage

The dynamic library can be used to access many add-on boards, the library must be initialized before connecting to the device and accessing the data, and library process function must be called cyclically to execute the command and communicate with the devices. There is a profile file which stored the configure information for the board, which must be imported during the library initialization. The flows are the logic schematic for your reference.



4.1API function list

Function Name	Description
Light Control Library	
Light Control Library Base	
NKLC_LibraryInit	Init the LC library and the drivers
NKLC_LibraryDeinit	Deinitialize the LC library and release the resources
Light Control Functions in async mode	
NKLC_OpenDevice_Async	Open the device port, for the light control
NKLC_CloseDevice_Async	Close and disconnect to the port
NKLC_IsDeviceOpened_Async	Check if the port is opened or not
NKLC_Process_Async	Notify function to process the command
NKLC_GetVerInfo_Async	Read the version information of the hardware and firmware
NKLC_SetPwmParams_Async	Send the parameters to turn on/off the specified light channel and the parameters will be saved to the controller.
NKLC_GetPwmParams_Async	Read the parameters of the specified light channel from the controller
NKLC_SetGeneralParam_Async	Set the general parameter for the controller
NKLC_GetGeneralParam_Async	Get the general parameter from the controller.
Light Control Functions	
NKLC_OpenDevice	Open the device port, for the light control
NKLC_CloseDevice	Close and disconnect to the port
NKLC_IsDeviceOpened	Check if the port is opened or not
NKLC_GetVerInfo	Read the version information of the hardware and firmware
NKLC_SetPwmParams	Send the parameters to turn on/off the specified light channel and the parameters will be saved to the controller.
NKLC_GetPwmParams	Read the parameters of the specified light channel from the controller
NKLC_SetGeneralParam	Set the general parameter for the controller
NKLC_GetGeneralParam	Get the general parameter from the controller.

4.2 Light Control Library

4.2.1 Light Control Library Base

4.2.1.1 NKLC_LibraryInit

- **Description**
LC Library initialized.
- **Prototype**
`int NKLC_LibraryInit(const char * configFile)`
- **Parmeters**
 - Input
 - ◆ `configIniFile`: the name and path of the configure file. The configuration for light control for each add-on board is stored in the ini format file, which can be got in the according folder after the SDK is installed.
 - Output
 - ◆ None
- **Return**
Return 0 if success; other return error codes.
- **Other**
The function should be called firstly but only once to allocate the resources before using the LC access functions in the library.
- **Usage**
`int ret = NKLC_LibraryInit (".nkio_config.ini");`

4.2.1.2 NKLC_LibraryDeinit

- **Description**
The function to be used to release the LC library resources before the application exit.
- **Prototype**
`void NKLC_LibraryDeinit(void)`
- **Parmeters**
 - Input
 - ◆ None
 - Output
 - ◆ None
- **Return**
None.
- **Other**
The function should be called before the application exit.
- **Usage**

```
int ret = NKLC_LibraryInit("nkio_config.ini");
if( ret == NKIO_ENOERR)
{
    While(1)
    {
        .....
        Sleep(1);
    }
}

NKLC_LibraryDeinit();
```


4.2.2 Light Control Functions in async mode

4.2.2.1 NKLC_OpenDevice_Async

■ Description

Send the command to open the port.

■ Prototype

```
int NKLC_OpenDevice_Async(unsigned short port, pLcCallbackFunc pCallBackFun)
```

■ Parameters

➤ Input

- ◆ port: the port to be opened, for the NP-61xx series products, COM3 is used to communicate with the light controller, so the port should be set to 3 when using the light controller. The serial port COM3 is mapped to /dev/ttyS2 under Linux systems, so the port should be set to 2 when using the light controller under Linux systems.
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure of parameters in the callback function as the below shows. The version of the hardware and firmware will be returned in the parameters when the device is opened successfully. Please remind that you can not execute the block code in the callback functions.

```
typedef struct _OPENCOM_CALLBACK_ARG_T
{
    unsigned short portNum;           // port number has been opened.
    unsigned char devId;              // the device Id opened
    unsigned char hardwareMajorVer;   // hardware major version returned
    unsigned char hardwareMinorVer;   // hardware minor version returned
    unsigned char hardwareRevVer;     // hardware release version returned
    unsigned char firmwareMajorVer;   // firmware major version returned
    unsigned char firmwareMinorVer;   // firmware minor version returned
    unsigned char firmwareRevVer;     // firmware release version returned
    unsigned char fillup_1;
    unsigned char fillup_2;
    unsigned char error;               // the error flag, 0:no error, and 1 when open error.
    unsigned int errorId;              // the error ID.
}OPENCOM_CALLBACK_ARG_T;
```

■ Usage

```
// Open port callback function
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
    else
    {
        printf("Hardware version: %d.%d.%d Firmware version: %d.%d.%d\n\r",
            arg.openComCallbackArg.hardwareMajorVer,
            arg.openComCallbackArg.hardwareMinorVer,
            arg.openComCallbackArg.hardwareRevVer,
            arg.openComCallbackArg.firmwareMajorVer,
            arg.openComCallbackArg.firmwareMinorVer,
            arg.openComCallbackArg.firmwareRevVer);
    }
}
```

```

// Server thread
DWORD WINAPI ServerThread(LPVOID lpParameter)
{
    While(1)
    {
        NKLC_Process_Async();
        Sleep(1);
    }
    return 0;
}

HANDLE xServerThreadHdl = NULL;
DWORD dwServerThreadId;
int main()
{
    int ret = NKLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        xServerThreadHdl = CreateThread(NULL, 1024, ServerThread, NULL, 0, & dwServerThreadId);
        if(NULL == xServerThreadHdl)
        {
            printf("Create server thread failed\n\r");
        }
        else
        {
            // open the port 3
            NKLC_OpenDevice_Async (3, LcOpenPortCallback);
        }
        .....
    }
    return 0;
}

```

4.2.2.2 NKLC_CloseDevice_Async

■ Description

Send the command to close the port.

■ Prototype

```
int NKLC_CloseDevice_Async(unsigned short port, pLcCallbackFunc pCallBackFun)
```

■ Parameters

➤ Input

- ◆ port: the port to be closed, for the NP-61xx series products, COM3 is used to communicate with the light controller, so the port should be set to 3.
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure output in the callback function as the below, please remind that you can not execute the block code in the callback functions.

```

typedef struct _CLOSECOM_CALLBACK_ARG_T
{
    unsigned int fill_up;
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error;      // error flag, 0: no error, 1: error happened
    unsigned int errorId;    // the error ID if the error happened.
}

```

```

        unsigned int fill_up_4;
    }CLOSECOM_CALLBACK_ARG_T;

```

■ Usage

// Open port callback function

```

void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        Printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
}

```

```

void LcClosePortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.closeComCallbackArg.error)
    {
        Printf("Close port failed: %d\n\r",
            arg.closeComCallbackArg.errorId);
    }
}

```

// Server thread

```

DWORD WINAPI ServerThread(LPVOID lpParameter)
{
    While(1)
    {
        NKLC_Process_Async();
        Sleep(1);
    }
    return 0;
}

```

HANDLE xServerThreadHdl = NULL;

DWORD dwServerThreadId;

int main()

```

{
    int ret = NKLC_LibraryInit(0);
    if( ret >= 0)
    {
        xServerThreadHdl = CreateThread(NULL,1024,ServerThread,NULL,0,& dwServerThreadId);
        if(NULL == xServerThreadHdl)
        {
            printf("Create server thread failed\n\r");
        }
        else
        {
            // open the port 3
            NKLC_OpenDevice_Async( 3, LcOpenPortCallback);

            .....

            // close the port if disconnect and exit the application
            NKLC_CloseDevice_Async(3, LcClosePortCallback);
        }
        .....
    }
    return 0;
}

```

4.2.2.3 NKLC_IsDeviceOpened_Async

- **Description**
Check if the port is opened.
- **Prototype**
`int NKLC_IsDeviceOpened_Async(unsigned short port, unsigned char devId)`
- **Parameters**
 - Input
 - ◆ port: the port number to be checked.
 - ◆ devId: the Id of the device, set to 0x01 for the light controller.
 - Output
 - ◆ None
- **Return**
Return 0 if not opened, otherwise return 1.
- **Other**
Only used to check the serial port.
- **Usage**
`int ret = NKLC_IsDeviceOpened_Async(3, 0x01); // check whether the COM3 is opened or not.`

4.2.2.4 NKLC_Process_Async

- **Description**
The notification to execute the command in the stack and communicate with the device by the serial port. This function must be called cyclically in the server thread.
- **Prototype**
`int NKLC_Process_Async(void)`
- **Parameters**
 - Input
 - ◆ None
 - Output
 - ◆ None
- **Return**
Return -1 if error, other value when success.
- **Other**
The function must be called cyclically in the server thread.
- **Usage**

```
int ret = NKLC_LibraryInit("nkio_config.ini");
if( ret >= 0)
.....
while(1)
{
    NKLC_Process_Async();
    Sleep(1);
}
```

4.2.2.5 NKLC_SetPwmParams_Async

- **Description**
The function is used to set the parameters to the specified light control channel. **And the parameters will be saved to the controller.**
- **Prototype**
`int NKLC_SetPwmParams_Async(unsigned int devId,
 unsigned char ucChIdx,
 unsigned char ucPwmMode,
 unsigned char ucPwmValue,
 unsigned char ucPwmHoldingTime,
 unsigned char ucPwmOnOff,
 pLcCallbackFunc pCallbackFun)`
- **Parameters**
 - Input
 - ◆ devId: the Id of the device, set to 0x01 for the light controller.
 - ◆ ucChIdx: the channel of the light to be set.
 - CH0: 0x01

-
- CH1: 0x02
 - CH2: 0x04
 - CH3: 0x08).
 - ◆ ucPwmMode: the light trigger mode.
 - 0: Soft switch, Control the light by the PwmOnOff value.
 - 1: Hard switch, triggered by the external sensor, the light is on when the trigger input is on, the light is off while the trigger input is off.
 - 2: External trigger on but delay off. The light is triggered when the trigger input on the rising edge, but the duration is set by the holdingtime.
 - 3: Soft Trigger, the light is on when turning on the light by the API, and will keep on with the holding time set, and then will be off automatically.
 - ◆ ucPwmValue: the brightness level of the light, from 0 to 100.
 - ◆ ucPwmHoldingTime: the duration time when the light working in the mode raising edge mode(2), it is meaningless in the other mode. Unit: second is default, but can be changed by the general parameters.
 - ◆ ucPwmOnOff: the switch of the light when working in the soft-trigger mode, set to 0 to turn off the light and set 1 to turn on the light, it is meaningless in the other trigger mode.
 - ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.
 - Output
 - ◆ None
 - **Return**
Return -1 if failed, and return the value larger than zero if success.
 - **Other**
The result will be output by the callback function, the structure as the below:

```
typedef struct _SET_PWM_PARAMS_CALLBACK_ARG_T
{
    unsigned int fill_up;
    unsigned char chIdx;      // the channel of the light has been set.
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char error;      // error flag, 0: no error, 1: error happened.
    unsigned int errorId;     // the error ID when the error happened.
    unsigned int fill_up_3;
}SET_PWM_PARAMS_CALLBACK_ARG_T;
```

■ Usage

// Open port callback function

```
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
```

```
{
    If (arg.openComCallbackArg.error)
    {
        Printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
}
```

```
void LcClosePortCallback(LC_CALLBACK_ARG_T arg)
```

```
{
    If (arg.closeComCallbackArg.error)
    {
        Printf("Close port failed: %d\n\r",
            arg.closeComCallbackArg.errorId);
    }
}
```

```
void LcSetPwmParamsCallback(LC_CALLBACK_ARG_T arg)
```

```
{
    If(arg.setPwmParamsCallbackArg.error)
    {
        printf("Set Pwm Params failed: %d\n\r", arg.setPwmParamsCallbackArg.errorId);
    }
}
```

```

    }
}

// Server thread
DWORD WINAPI ServerThread(LPVOID lpParameter)
{
    While(1)
    {
        NKLC_Process_Async();
        Sleep(1);
    }
    return 0;
}

HANDLE xServerThreadHdl = NULL;
DWORD dwServerThreadId;
int main()
{
    int ret = NKLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        xServerThreadHdl = CreateThread(NULL, 1024, ServerThread, NULL, 0, & dwServerThreadId);
        if(NULL == xServerThreadHdl)
        {
            printf("Create server thread failed\n\r");
        }
        else
        {
            // open the port 3
            NKLC_OpenDevive_Async(3, LcOpenPortCallback);

            // send the command to set the parameters for the CH0
            NKLC_SetPwmParams_Async(0x01,
                                    0x01, // set params to CH0
                                    0x0,  // soft-trigger mode
                                    80,    // the brightness level is 80%
                                    5,     // make no sense in the current mode
                                    1,     // turn on the light using the current parameters
                                    LcSetPwmParamsCallback);

            .....

            // close the port if disconnect and exit the application
            NKLC_CloseDevice_Async(3, LcClosePortCallback);
        }
        .....
    }
    return 0;
}

```

4.2.2.6 NKLC_GetPwmParams_Async

■ Description

The function is used to get the parameters of the specified light control channel.

■ Prototype

```
int NKLC_GetPwmParams_Async(unsigned int devId,
                           unsigned char ucChIdx,
                           pLcCallbackFunc pCallBackFun)
```

■ Parameters

➤ Input

- ◆ devId: the Id of the device, set to 0x01 for the light controller.
- ◆ ucChIdx: the channel of the light to be set.
 - CH0: 0x01

-
- CH1: 0x02
 - CH2: 0x04
 - CH3: 0x08).
 - ◆ pCallbackFun: the pointer of the callback function, the function will be called once after the command is executed. All of the parameters will be output in the callback function.
 - Output
 - ◆ None
 - **Return**
Return -1 if failed, and return the value larger than zero if success.
 - **Other**
The result will be output by the callback function, the structure as the below:

```
typedef struct _GET_PWM_PARAMS_CALLBACK_ARG_T
{
    unsigned char chIdx; // the specified light channel, CH0: 0x1, CH1: 0x2, CH2: 0x4, CH3: 0x8
    unsigned char pwmMode; // trigger mode, 0:Soft-switch, 1: hardware-switch, 2: Raising-Edge,
    3: Soft-trigger
    unsigned char pwmValue; // Current brightness level
    unsigned char pwmHoldingTime; // the holding time
    unsigned char pwmOnOff; // the status of the light, 0: off, 1: on
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error; // error flag, 0: no error, 1: error happened.
    unsigned int errorId; // error ID when the error happened.
    unsigned int fill_up_4;
}GET_PWM_PARAMS_CALLBACK_ARG_T;
```

■ Usage

// Open port callback function

```
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
```

```
{
    If (arg.openComCallbackArg.error)
    {
        Printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
}
```

```
void LcClosePortCallback(LC_CALLBACK_ARG_T arg)
```

```
{
    If (arg.closeComCallbackArg.error)
    {
        Printf("Close port failed: %d\n\r",
            arg.closeComCallbackArg.errorId);
    }
}
```

```
void LcGetPwmParamsCallback(LC_CALLBACK_ARG_T arg)
```

```
{
    If(arg.getPwmParamsCallbackArg.error)
    {
        printf("get Pwm Params failed: %d\n\r", arg.getPwmParamsCallbackArg.errorId);
    }
    else
    {
        printf("CH%d: Mode:%d, value: %d, HoldingTime: %d, Status: %d\n\r",
            arg.getPwmParamsCallbackArg.chIdx,
            arg.getPwmParamsCallbackArg.pwmMode,
            arg.getPwmParamsCallbackArg.pwmValue,
            arg.getPwmParamsCallbackArg.pwmHoldingTime,
            arg.getPwmParamsCallbackArg.pwmOnOff);
    }
}
```

```

}

// Server thread
DWORD WINAPI ServerThread(LPVOID lpParameter)
{
    While(1)
    {
        NKLC_Process_Async();
        Sleep(1);
    }
    return 0;
}

HANDLE xServerThreadHdl = NULL;
DWORD dwServerThreadId;
int main()
{
    int ret = NKLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        xServerThreadHdl = CreateThread(NULL, 1024, ServerThread, NULL, 0, & dwServerThreadId);
        if(NULL == xServerThreadHdl)
        {
            printf("Create server thread failed\n\r");
        }
        else
        {
            // open the port 3
            NKLC_OpenDevice_Async( 3, LcOpenPortCallback);

            // send the command to get the parameters of the CH1
            NKLC_GetPwmParams_Async(0x01,
                                    0x02,
                                    LcGetPwmParamsCallback);
            .....

            // close the port if disconnect and exit the application
            NKLC_CloseDevice_Async( 3, LcClosePortCallback);
        }
        .....
    }
    return 0;
}

```

4.2.2.7 NKLC_SetGeneralParam_Async

■ Description

Set general parameters to the controller, please refer to the parameter table supported.

■ Prototype

```
int NKLC_SetGeneralParam_Async(unsigned int devId,
                               unsigned char ucParamId,
                               unsigned char ucParamLen,
                               unsigned int uiParamValue,
                               pLcCallbackFunc pCallBackFun)
```

■ Parameters

➤ Input

- ◆ devId: the Id of the device, set to 0x01 for the light controller.
- ◆ ucParamId: the ID of the parameter to be set.
- ◆ ucParamLen: the length of the parameter to be set.
- ◆ uiParamValue: the value of the parameter to be set.
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.

- Output
 - ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure of parameters in the callback function as the below shows. The parameter information will be returned in the call back function once the parameter is set successfully. Please remind that you can not execute the block code in the callback functions.

```
typedef struct _SET_GENERAL_PARAM_CALLBACK_ARG_T
{
    unsigned char devId;           // The ID of the device, 0x01 is for the NP add-on IO board.
    unsigned char cmdId;          // Ignore.
    unsigned char paramId;        // The parameter ID should be set in the controller.
    unsigned char paramLen;       // The length of the parameter value in the controller.
    unsigned int paramValue;      // The value of the parameter has been set.
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error;          // Error flag when set failed, 0 is successful.
    unsigned int errorId;         // The error ID when the error happened.
}SET_GENERAL_PARAM_CALLBACK_ARG_T;
```

■ Parameter table

ID	Parameter	Value	R/W	Length(Byte)	Description
1	Device ID	1~255	R	1 Byte	For the light control IO board, it is 1.
2	Hardware version Major	0~255	R	1 Byte	
3	Hardware version Minor	0~255	R	1 Byte	
4	Hardware version Release	0~255	R	1 Byte	
5	Firmware version Major	0~255	R	1 Byte	
6	Firmware version Minor	0~255	R	1 Byte	
7	Firmware version Release	0~255	R	1 Byte	
10	Light Control Channel 0 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
11	Light Control Channel 1 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
12	Light Control Channel 2 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
13	Light Control Channel 3 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.

Table 1 General parameter

■ Usage

// Set the general parameter callback.

```
void SetGeneralParameterCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.setGeneralParamCallbackArg.error)
    {
        printf("Set general parameter %d to %d failed\n\r",
            arg.setGeneralParamCallbackArg.paramId,
            arg.setGeneralParamCallbackArg.paramValue);
    }
    else
    {
        printf("Set general parameter %d to %d success\n\r",
            arg.setGeneralParamCallbackArg.paramId,
            arg.setGeneralParamCallbackArg.paramValue);
    }
}
```

// Open port callback function

```
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
    else
    {
        printf("Hardware version: %d.%d.%d Firmware version: %d.%d.%d\n\r",
            arg.openComCallbackArg.hardwareMajorVer,
            arg.openComCallbackArg.hardwareMinorVer,
            arg.openComCallbackArg.hardwareRevVer,
            arg.openComCallbackArg.firmwareMajorVer,
            arg.openComCallbackArg.firmwareMinorVer,
            arg.openComCallbackArg.firmwareRevVer);
    }
}
```

// Server thread

```
DWORD WINAPI ServerThread(LPVOID lpParameter)
```

```
{
    While(1)
    {
        NKLC_Process_Async();
        Sleep(1);
    }
    return 0;
}
```

```
HANDLE xServerThreadHdl = NULL;
```

```
DWORD dwServerThreadId;
```

```
int main()
```

```
{
    int ret = NKLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        xServerThreadHdl = CreateThread(NULL,1024,ServerThread,NULL,0,& dwServerThreadId);
        if(NULL == xServerThreadHdl)
        {
            printf("Create server thread failed\n\r");
        }
    }
}
```

```

else
{
    // open the port 3
    NKLC_OpenDevice_Async(3, LcOpenPortCallback);
    .....
    // Set the holding time unit of the channel 0 to 100ms
    NKLC_SetGeneralParam_Async(0x01, // DevId
                                10, // Parameter ID
                                1, // Parameter Length
                                1, // Parameter Value
                                SetGeneralParameterCallback // Operate call back
                                );
    .....
}
.....
}
return 0;
}

```

4.2.2.8 NKLC_GetGeneralParam_Async

■ Description

Get general parameters from the controller, please refer to the parameter table supported.

■ Prototype

```

int NKLC_GetGeneralParam_Async(unsigned int devId,
    unsigned char ucParamId,
    unsigned char ucParamLen,
    pLcCallbackFunc pCallbackFun)

```

■ Parameters

➤ Input

- ◆ devId: the Id of the device, set to 0x01 for the light controller.
- ◆ ucParamId: the ID of the parameter to be read.
- ◆ ucParamLen: the length of the parameter to be read.
- ◆ pCallbackFun: the pointer of the callback function, the function will be called once after the command is executed.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure of parameters in the callback function as the below shows. The parameter information will be returned in the callback function once the parameter is read successfully. Please remind that you can not execute the block code in the callback functions.

```

typedef struct _GET_GENERAL_PARAM_CALLBACK_ARG_T
{
    unsigned char devId; // The ID of the device, 0x01 is for the NP add-on IO board.
    unsigned char cmdId; // Ignore.
    unsigned char paramId; // The parameter ID should be got from the controller.
    unsigned char paramLen; // The length of the parameter value.
    unsigned int paramValue; // The value of the parameter has been got.
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error; // Error flag when set failed, 0 is successful.
    unsigned int errorId; // The error ID when the error happened.
}GET_GENERAL_PARAM_CALLBACK_ARG_T;

```

■ Parameter table

ID	Parameter	Value	R/W	Length(Byte)	Description
1	Device ID	1~255	R	1 Byte	For the light control IO board, it is 1.
2	Hardware version Major	0~255	R	1 Byte	
3	Hardware version Minor	0~255	R	1 Byte	
4	Hardware version Release	0~255	R	1 Byte	
5	Firmware version Major	0~255	R	1 Byte	
6	Firmware version Minor	0~255	R	1 Byte	
7	Firmware version Release	0~255	R	1 Byte	
10	Light Control Channel 0 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
11	Light Control Channel 1 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
12	Light Control Channel 2 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
13	Light Control Channel 3 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.

Table 2 General parameter

■ Usage

// Get the general parameter callback.

```
void GetGeneralParameterCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.getGeneralParamCallbackArg.error)
    {
        printf("Get general parameter %d failed\n\r",
            arg.getGeneralParamCallbackArg.paramId);
    }
    else
    {
        printf("Get general parameter %d = %d success\n\r",
            arg.getGeneralParamCallbackArg.paramId,
            arg.getGeneralParamCallbackArg.paramValue);
    }
}
```

// Open port callback function

```
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        printf("Open port %d failed: %d\n\r",
```

```

        arg.openComCallbackArg.portNum,
        arg.openComCallbackArg.errorId);
    }
    else
    {
        printf("Hardware version: %d.%d.%d Firmware version: %d.%d.%d\n\r",
            arg.openComCallbackArg.hardwareMajorVer,
            arg.openComCallbackArg.hardwareMinorVer,
            arg.openComCallbackArg.hardwareRevVer,
            arg.openComCallbackArg.firmwareMajorVer,
            arg.openComCallbackArg.firmwareMinorVer,
            arg.openComCallbackArg.firmwareRevVer);
    }
}

// Server thread
DWORD WINAPI ServerThread(LPVOID lpParameter)
{
    While(1)
    {
        DIOLC_Process();
        Sleep(1);
    }
    return 0;
}

HANDLE xServerThreadHdl = NULL;
DWORD dwServerThreadId;
int main()
{
    int ret = DIOLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        xServerThreadHdl = CreateThread(NULL, 1024, ServerThread, NULL, 0, & dwServerThreadId);
        if(NULL == xServerThreadHdl)
        {
            printf("Create server thread failed\n\r");
        }
        else
        {
            // open the port 3
            NKLC_OpenDevice_Async (3, LcOpenPortCallback);
            .....
            // Get the holding time unit of the channel 0
            NKLC_GetGeneralParam_Async(0x01, // DevId
                                      10, // Parameter ID
                                      1, // Parameter Length
                                      GetGeneralParameterCallback // Operate call back
                                      );
            .....
        }
        .....
    }
    return 0;
}

```

4.2.3 Light Control Functions

4.2.3.1 NKLC_OpenDevice

■ Description

Send the command to open the port.

■ Prototype

```
int NKLC_OpenDevice(unsigned short port, pLcCallbackFunc pCallBackFun)
```

■ Parameters

➤ Input

- ◆ port: the port to be opened, for the NP-61xx series products, COM3 is used to communicate with the light controller, so the port should be set to 3 when using the light controller. The serial port COM3 is mapped to /dev/ttyS2 under Linux systems, so the port should be set to 2 when using the light controller under Linux systems.
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure of parameters in the callback function as the below shows. The version of the hardware and firmware will be returned in the parameters when the device is opened successfully. Please remind that you can not execute the block code in the callback functions.

```
typedef struct _OPENCOM_CALLBACK_ARG_T
{
    unsigned short portNum;           // port number has been opened.
    unsigned char devId;              // the device Id opened
    unsigned char hardwareMajorVer;   // hardware major version returned
    unsigned char hardwareMinorVer;   // hardware minor version returned
    unsigned char hardwareRevVer;     // hardware release version returned
    unsigned char firmwareMajorVer;   // firmware major version returned
    unsigned char firmwareMinorVer;   // firmware minor version returned
    unsigned char firmwareRevVer;     // firmware release version returned
    unsigned char fillup_1;
    unsigned char fillup_2;
    unsigned char error;               // the error flag, 0:no error, and 1 when open error.
    unsigned int errorId;              // the error ID.
}OPENCOM_CALLBACK_ARG_T;
```

■ Usage

```
// Open port callback function
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
    else
    {
        printf("Hardware version: %d.%d.%d Firmware version: %d.%d.%d\n\r",
            arg.openComCallbackArg.hardwareMajorVer,
            arg.openComCallbackArg.hardwareMinorVer,
            arg.openComCallbackArg.hardwareRevVer,
            arg.openComCallbackArg.firmwareMajorVer,
            arg.openComCallbackArg.firmwareMinorVer,
            arg.openComCallbackArg.firmwareRevVer);
    }
}
```

```

int main()
{
    int ret = NKLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        // open the port 3
        NKLC_OpenDevice (3, LcOpenPortCallback);
        .....
    }
    return 0;
}

```

4.2.3.2 NKLC_CloseDevice

■ Description

Send the command to close the port.

■ Prototype

```
int NKLC_CloseDevice(unsigned short port, pLcCallbackFunc pCallBackFun)
```

■ Parameters

➤ Input

- ◆ port: the port to be closed, for the NP-61xx series products, COM3 is used to communicate with the light controller, so the port should be set to 3.
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure output in the callback function as the below, please remind that you can not execute the block code in the callback functions.

```

typedef struct _CLOSECOM_CALLBACK_ARG_T
{
    unsigned int fill_up;
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error;        // error flag, 0: no error, 1: error happened
    unsigned int errorId;       // the error ID if the error happened.
    unsigned int fill_up_4;
}CLOSECOM_CALLBACK_ARG_T;

```

■ Usage

```

// Open port callback function
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        Printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
}

void LcClosePortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.closeComCallbackArg.error)
    {
        Printf("Close port failed: %d\n\r",
            arg.closeComCallbackArg.errorId);
    }
}

```

```

    }
}

int main()
{
    int ret = NKLC_LibraryInit(0);
    if( ret >= 0)
    {
        // open the port 3
        NKLC_OpenDevice( 3, LcOpenPortCallback);

        .....

        // close the port if disconnect and exit the application
        NKLC_CloseDevice(3, LcClosePortCallback);
        .....
    }
    return 0;
}

```

4.2.3.3 NKLC_IsDeviceOpened

- **Description**
Check if the port is opened.
- **Prototype**
int NKLC_IsDeviceOpened(unsigned short port, unsigned char devId)
- **Parameters**
 - Input
 - ◆ port: the port number to be checked.
 - ◆ devId: the Id of the device, set to 0x01 for the light controller.
 - Output
 - ◆ None
- **Return**
Return 0 if not opened, otherwise return 1.
- **Other**
Only used to check the serial port.
- **Usage**
int ret = NKLC_IsDeviceOpened(3, 0x01); // check whether the COM3 is opened or not.

4.2.3.4 NKLC_SetPwmParams

- **Description**
The function is used to set the parameters to the specified light control channel. **And the parameters will be saved to the controller.**
- **Prototype**
int NKLC_SetPwmParams(unsigned int devId,
 unsigned char ucChIdx,
 unsigned char ucPwmMode,
 unsigned char ucPwmValue,
 unsigned char ucPwmHoldingTime,
 unsigned char ucPwmOnOff,
 pLcCallbackFunc pCallbackFun)
- **Parameters**
 - Input
 - ◆ devId: the Id of the device, set to 0x01 for the light controller.
 - ◆ ucChIdx: the channel of the light to be set.
 - CH0: 0x01
 - CH1: 0x02
 - CH2: 0x04
 - CH3: 0x08).
 - ◆ ucPwmMode: the light trigger mode.
 - 0: Soft switch, Control the light by the PwmOnOff value.

- 1: Hard switch, triggered by the external sensor, the light is on when the trigger input is on, the light is off while the trigger input is off.
 - 2: External trigger on but delay off. The light is triggered when the trigger input on the rising edge, but the duration is set by the holdingtime.
 - 3: Soft Trigger, the light is on when turning on the light by the API, and will keep on with the holding time set, and then will be off automatically.
 - ◆ ucPwmValue: the brightness level of the light, from 0 to 100.
 - ◆ ucPwmHoldingTime: the duration time when the light working in the mode raising edge mode(2), it is meaningless in the other mode. Unit: second is default, but can be changed by the general parameters.
 - ◆ ucPwmOnOff: the switch of the light when working in the soft-trigger mode, set to 0 to turn off the light and set 1 to turn on the light, it is meaningless in the other trigger mode.
 - ◆ pCallbackFun: the pointer of the callback function, the function will be called once after the command is executed.
- Output
- ◆ None
- **Return**
Return -1 if failed, and return the value larger than zero if success.
- **Other**
The result will be output by the callback function, the structure as the below:

```
typedef struct _SET_PWM_PARAMS_CALLBACK_ARG_T
{
    unsigned int fill_up;
    unsigned char chldx;    // the channel of the light has been set.
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char error;    // error flag, 0: no error, 1: error happened.
    unsigned int errorId;    // the error ID when the error happened.
    unsigned int fill_up_3;
}SET_PWM_PARAMS_CALLBACK_ARG_T;
```

■ **Usage**

```
// Open port callback function
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        Printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
}

void LcClosePortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.closeComCallbackArg.error)
    {
        Printf("Close port failed: %d\n\r",
            arg.closeComCallbackArg.errorId);
    }
}

void LcSetPwmParamsCallback(LC_CALLBACK_ARG_T arg)
{
    If(arg.setPwmParamsCallbackArg.error)
    {
        printf("Set Pwm Params failed: %d\n\r", arg.setPwmParamsCallbackArg.errorId);
    }
}

int main()
{
```

```

int ret = NKLC_LibraryInit("./nkio_config.ini");
if( ret >= 0)
{

    // open the port 3
    NKLC_OpenDevive(3, LcOpenPortCallback);

    // send the command to set the parameters for the CH0
    NKLC_SetPwmParams(0x01,
        0x01,        // set params to CH0
        0x0,         // soft-trigger mode
        80,          // the brightness level is 80%
        5,           // make no sense in the current mode
        1,           // turn on the light using the current parameters
        LcSetPwmParamsCallback);

    .....

    // close the port if disconnect and exit the application
    NKLC_CloseDevice(3, LcClosePortCallback);

    .....
}
return 0;
}

```

4.2.3.5 NKLC_GetPwmParams

■ Description

The function is used to get the parameters of the specified light control channel.

■ Prototype

```

int NKLC_GetPwmParams(unsigned int devId,
    unsigned char ucChIdx,
    pLcCallbackFunc pCallBackFun)

```

■ Parameters

➤ Input

- ◆ devId: the Id of the device, set to 0x01 for the light controller.
- ◆ ucChIdx: the channel of the light to be set.
 - CH0: 0x01
 - CH1: 0x02
 - CH2: 0x04
 - CH3: 0x08).
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed. All of the parameters will be output in the callback function.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the value larger than zero if success.

■ Other

The result will be output by the callback function, the structure as the below:

```

typedef struct _GET_PWM_PARAMS_CALLBACK_ARG_T
{
    unsigned char chIdx; // the specified light channel, CH0: 0x1, CH1: 0x2, CH2: 0x4, CH3: 0x8
    unsigned char pwmMode; // trigger mode, 0:Soft-switch, 1: hardware-switch, 2: Raising-Edge,
    3: Soft-trigger
    unsigned char pwmValue; // Current brightness level
    unsigned char pwmHoldingTime; // the holding time
    unsigned char pwmOnOff; // the status of the light, 0: off, 1: on
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error; // error flag, 0: no error, 1: error happened.
    unsigned int errorId; // error ID when the error happened.
    unsigned int fill_up_4;
}

```

```

    }GET_PWM_PARAMS_CALLBACK_ARG_T;
■ Usage
// Open port callback function
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        Printf("Open port %d failed: %d\n\r",
               arg.openComCallbackArg.portNum,
               arg.openComCallbackArg.errorId);
    }
}

void LcClosePortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.closeComCallbackArg.error)
    {
        Printf("Close port failed: %d\n\r",
               arg.closeComCallbackArg.errorId);
    }
}

void LcGetPwmParamsCallback(LC_CALLBACK_ARG_T arg)
{
    If(arg.getPwmParamsCallbackArg.error)
    {
        printf("get Pwm Params failed: %d\n\r", arg.getPwmParamsCallbackArg.errorId);
    }
    else
    {
        printf("CH%d: Mode:%d, value: %d, HoldingTime: %d, Status: %d\n\r",
               arg.getPwmParamsCallbackArg.chIdx,
               arg.getPwmParamsCallbackArg.pwmMode,
               arg.getPwmParamsCallbackArg.pwmValue,
               arg.getPwmParamsCallbackArg.pwmHoldingTime,
               arg.getPwmParamsCallbackArg.pwmOnOff);
    }
}

int main()
{
    int ret = NKLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        // open the port 3
        NKLC_OpenDevice( 3, LcOpenPortCallback);

        // send the command to get the parameters of the CH1
        NKLC_GetPwmParams(0x01,
                           0x02,
                           LcGetPwmParamsCallback);
        .....

        // close the port if disconnect and exit the application
        NKLC_CloseDevice( 3, LcClosePortCallback);
        .....
    }
    return 0;
}

```

4.2.3.6 NKLC_SetGeneralParam

■ Description

Set general parameters to the controller, please refer to the parameter table supported.

■ Prototype

```
int NKLC_SetGeneralParam(unsigned int devId,
                        unsigned char ucParamId,
                        unsigned char ucParamLen,
                        unsigned int uiParamValue,
                        pLcCallbackFunc pCallBackFun)
```

■ Parameters

➤ Input

- ◆ devId: the Id of the device, set to 0x01 for the light controller.
- ◆ ucParamId: the ID of the parameter to be set.
- ◆ ucParamLen: the length of the parameter to be set.
- ◆ uiParamValue: the value of the parameter to be set.
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure of parameters in the callback function as the below shows. The parameter information will be returned in the call back function once the parameter is set successfully. Please remind that you can not execute the block code in the callback functions.

```
typedef struct _SET_GENERAL_PARAM_CALLBACK_ARG_T
{
    unsigned char devId;           // The ID of the device, 0x01 is for the NP add-on IO board.
    unsigned char cmdId;          // Ignore.
    unsigned char paramId;        // The parameter ID should be set in the controller.
    unsigned char paramLen;       // The length of the parameter value in the controller.
    unsigned int paramValue;      // The value of the parameter has been set.
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error;          // Error flag when set failed, 0 is successful.
    unsigned int errorId;         // The error ID when the error happened.
}SET_GENERAL_PARAM_CALLBACK_ARG_T;
```

■ Parameter table

ID	Parameter	Value	R/W	Length(Byte)	Description
1	Device ID	1~255	R	1 Byte	For the light control IO board, it is 1.
2	Hardware version Major	0~255	R	1 Byte	
3	Hardware version Minor	0~255	R	1 Byte	
4	Hardware version Release	0~255	R	1 Byte	
5	Firmware version Major	0~255	R	1 Byte	
6	Firmware version Minor	0~255	R	1 Byte	
7	Firmware version Release	0~255	R	1 Byte	
10	Light Control Channel 0 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.

11	Light Control Channel 1 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
12	Light Control Channel 2 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
13	Light Control Channel 3 holding time unit	0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.

Table 1 General parameter

■ Usage

// Set the general parameter callback.

```
void SetGeneralParameterCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.setGeneralParamCallbackArg.error)
    {
        printf("Set general parameter %d to %d failed\n\r",
            arg.setGeneralParamCallbackArg.paramId,
            arg.setGeneralParamCallbackArg.paramValue);
    }
    else
    {
        printf("Set general parameter %d to %d success\n\r",
            arg.setGeneralParamCallbackArg.paramId,
            arg.setGeneralParamCallbackArg.paramValue);
    }
}
```

// Open port callback function

```
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
    else
    {
        printf("Hardware version: %d.%d.%d Firmware version: %d.%d.%d\n\r",
            arg.openComCallbackArg.hardwareMajorVer,
            arg.openComCallbackArg.hardwareMinorVer,
            arg.openComCallbackArg.hardwareRevVer,
            arg.openComCallbackArg.firmwareMajorVer,
            arg.openComCallbackArg.firmwareMinorVer,
            arg.openComCallbackArg.firmwareRevVer);
    }
}
```

```
int main()
{
    int ret = NKLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        // open the port 3
    }
}
```

```

NKLC_OpenDevice(3, LcOpenPortCallback);
.....
// Set the holding time unit of the channel 0 to 100ms
NKLC_SetGeneralParam(0x01, // DevId
                     10, // Parameter ID
                     1, // Parameter Length
                     1, // Parameter Value
                     SetGeneralParameterCallback // Operate call back
                     );

.....
.....
}
return 0;
}

```

4.2.3.7 NKLC_GetGeneralParam

■ Description

Get general parameters from the controller, please refer to the parameter table supported.

■ Prototype

```

int NKLC_GetGeneralParam(unsigned int devId,
                        unsigned char ucParamId,
                        unsigned char ucParamLen,
                        pLcCallbackFunc pCallBackFun)

```

■ Parameters

➤ Input

- ◆ devId: the Id of the device, set to 0x01 for the light controller.
- ◆ ucParamId: the ID of the parameter to be read.
- ◆ ucParamLen: the length of the parameter to be read.
- ◆ pCallBackFun: the pointer of the callback function, the function will be called once after the command is executed.

➤ Output

- ◆ None

■ Return

Return -1 if failed, and return the other value if success.

■ Other

The structure of parameters in the callback function as the below shows. The parameter information will be returned in the callback function once the parameter is read successfully. Please remind that you can not execute the block code in the callback functions.

```

typedef struct _GET_GENERAL_PARAM_CALLBACK_ARG_T
{
    unsigned char devId; // The ID of the device, 0x01 is for the NP add-on IO board.
    unsigned char cmdId; // Ignore.
    unsigned char paramId; // The parameter ID should be got from the controller.
    unsigned char paramLen; // The length of the parameter value.
    unsigned int paramValue; // The value of the parameter has been got.
    unsigned char fill_up_1;
    unsigned char fill_up_2;
    unsigned char fill_up_3;
    unsigned char error; // Error flag when set failed, 0 is successful.
    unsigned int errorId; // The error ID when the error happened.
}GET_GENERAL_PARAM_CALLBACK_ARG_T;

```

■ Parameter table

ID	Parameter	Value	R/W	Length(Byte)	Description
1	Device ID	1~255	R	1 Byte	For the light control IO board, it is 1.

2	Hardware version	Major	0~255	R	1 Byte	
3	Hardware version	Minor	0~255	R	1 Byte	
4	Hardware version	Release	0~255	R	1 Byte	
5	Firmware version	Major	0~255	R	1 Byte	
6	Firmware version	Minor	0~255	R	1 Byte	
7	Firmware version	Release	0~255	R	1 Byte	
10	Light Control Channel 0 holding time unit		0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
11	Light Control Channel 1 holding time unit		0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
12	Light Control Channel 2 holding time unit		0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.
13	Light Control Channel 3 holding time unit		0: 1000ms 1: 100ms 2: 10ms 3: 1ms	R/W	1 Byte	Only valid in the edge triggering mode.

Table 2 General parameter

■ Usage

// Get the general parameter callback.

```
void GetGeneralParameterCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.getGeneralParamCallbackArg.error)
    {
        printf("Get general parameter %d failed\n\r",
            arg.getGeneralParamCallbackArg.paramId);
    }
    else
    {
        printf("Get general parameter %d = %d success\n\r",
            arg.getGeneralParamCallbackArg.paramId,
            arg.getGeneralParamCallbackArg.paramValue);
    }
}
```

// Open port callback function

```
void LcOpenPortCallback(LC_CALLBACK_ARG_T arg)
{
    If (arg.openComCallbackArg.error)
    {
        printf("Open port %d failed: %d\n\r",
            arg.openComCallbackArg.portNum,
            arg.openComCallbackArg.errorId);
    }
    else
    {

```

```

        printf("Hardware version: %d.%d.%d Firmware version: %d.%d.%d\n\r",
               arg.openComCallbackArg.hardwareMajorVer,
               arg.openComCallbackArg.hardwareMinorVer,
               arg.openComCallbackArg.hardwareRevVer,
               arg.openComCallbackArg.firmwareMajorVer,
               arg.openComCallbackArg.firmwareMinorVer,
               arg.openComCallbackArg.firmwareRevVer);
    }
}

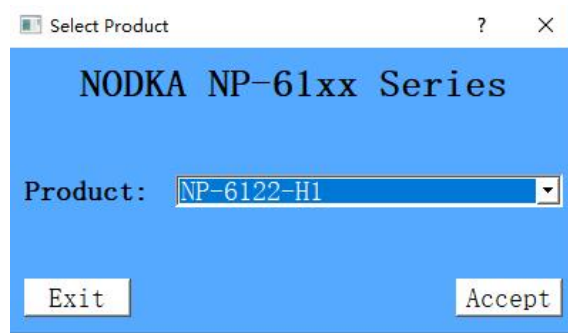
int main()
{
    int ret = DIOLC_LibraryInit("./nkio_config.ini");
    if( ret >= 0)
    {
        // open the port 3
        NKLC_OpenDevice (3, LcOpenPortCallback);
        .....
        // Get the holding time unit of the channel 0
        NKLC_GetGeneralParam(0x01, // DevId
                             10, // Parameter ID
                             1, // Parameter Length
                             GetGeneralParameterCallback // Operate call back
                             );
        .....
    }
    return 0;
}

```


5Test Tool

Target to easily check the IO board firstly before your development, Nodka provide a small tool called NKIO_TEST_TOOL.exe which can be used to check the IO working status, it is also can be used to configure some parameters when the IO light controller is working in the external trigger mode. Furthermore, the tool also provides the interface to update the firmware if necessary. The usage of the test tool will be illustrated in this chapter.

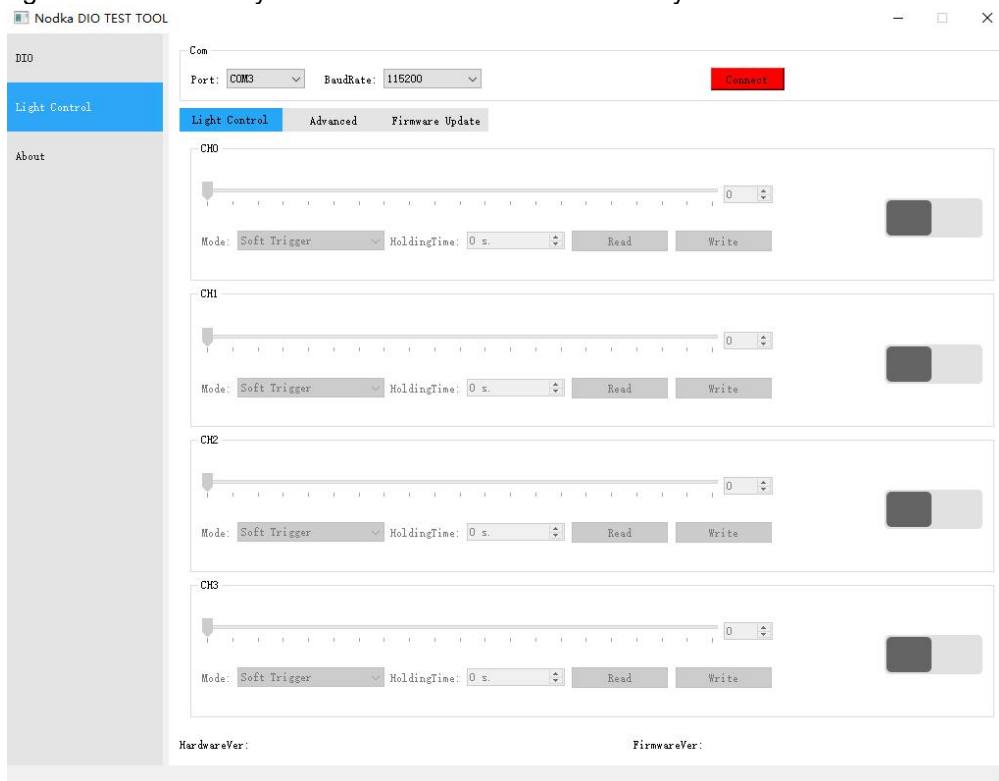
Since the test tool can be used for different I/O addon boards by importing the configuration file, so you must select correct board in the below dialog to import the according profile into the driver.



5.1Light Control Test

The PC communicate with the light controller by the serial port 3 in default (Baudrate: 115200, 8 databits, 1 stopbit, no parity) in the current solution.

So the port num is fixed to be COM3 in the current tool, the background color of the connect button will be red if not connected, and will be green when connect successfully. All of the control in the current page are available only in the case of connect successfully.

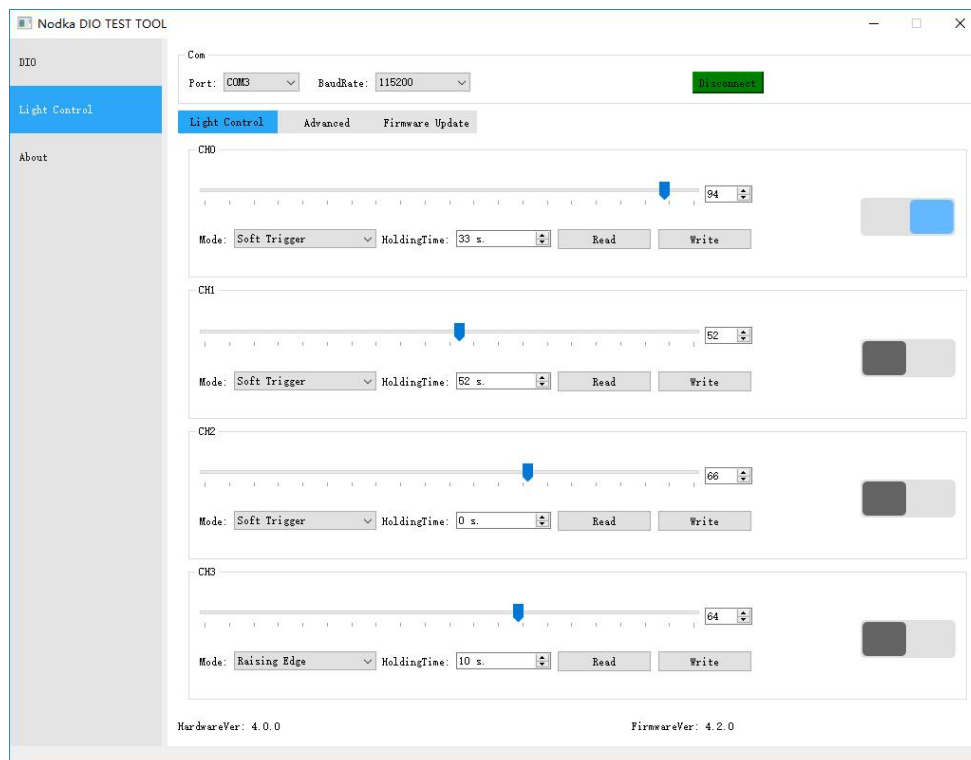


Once the connection is built, the device version information will be displayed, and this tool can only be used on the hardware version 4.0.0 or newer. For the other version, please contact the Nodka support team or reseller to get the detailed information.

In the light control tab, the parameters for each channel will be uploaded in the tool after the connection is built, then parameters for each independent channel can be changed and write into the controller by press the write button or by turning on the switch button on the right in each channel group.

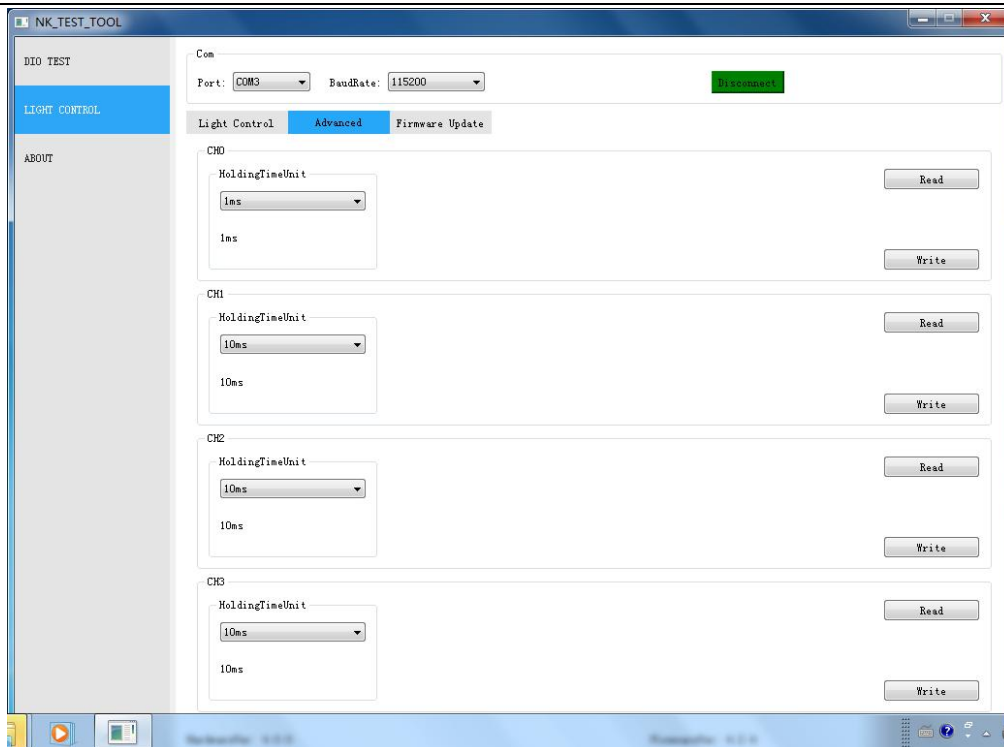
When the lighting controller working in the soft-trigger mode, the switch button can be used to turn on or off the according light channel, and even the brightness level(0~100) can be changed dynamically by the slider bar or spin box when the light is on. The switch button makes no sense in the other hard trigger mode or raising edge mode. The following are the description for the light channels.

- ◆ Mode: the light trigger mode.
 - 0: Soft Switch, set the brightness of the light by the PWM value.
 - 1: Hard Switch, triggered by the external sensor, the light is on when the sensor is on, the light is off while the sensor is off.
 - 2: Raising Edge. The light is triggered when the sensor on the rising edge, but the duration is set by the holdingtime to be set.
- ◆ HoldingTime: the duration time when the light working in the mode raising edge mode(2), it is meaningless in the other mode. Unit: default is second, which can be set from firmware version 4.2.4.



5.1.1 Advanced

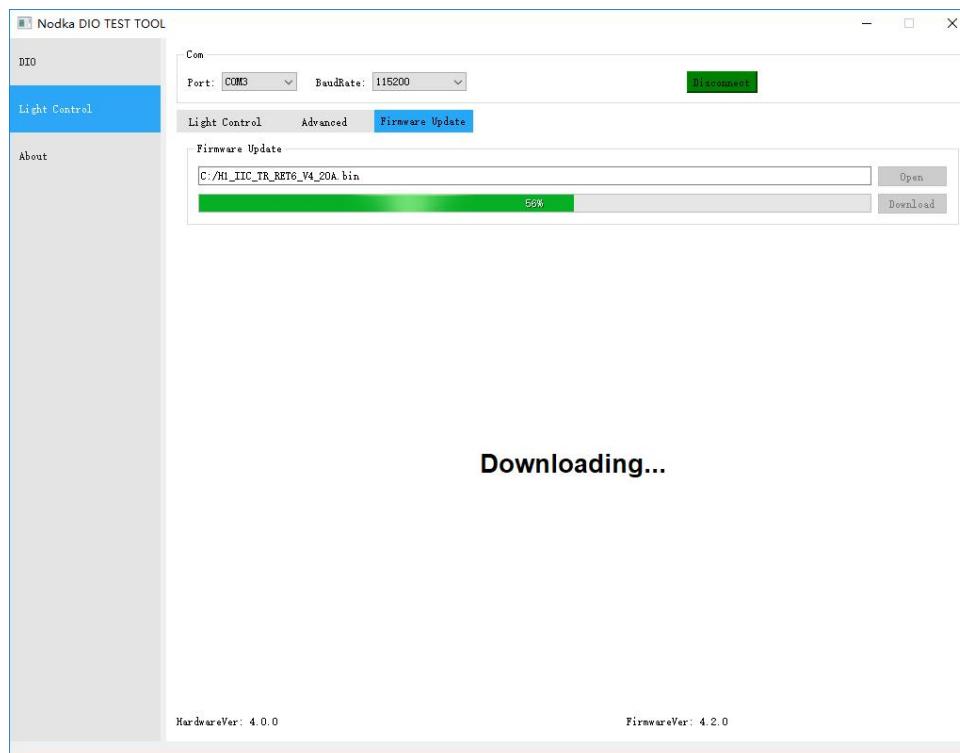
In the advanced page, you can get and set the advanced parameters for the light controller. From the firmware 4.2.4, the holding time unit can be set in the advanced page.



5.2 Firmware update

Nodka add-on board also support the IAP functionality to update the firmware of the controller on the line if necessary.

After the serial communication is connected, change to firmware Update tab, you can select the image file on the PC by pressing the open button, then start to download the image file into the controller by pressing the download button. The progress bar will display the downloading progress, and once finished, the open button will be activated again, and the controller will reboot to use the latest firmware fastly.



Please remind to check the hardware version information and ask for support from Nodka to get the correct firmware image file.

6Development

Nodka provide the dynamic library for the user to develop the programs. You can find the library and the header files in the SDK installation path. Please add the the files in the library folder into the same path with your own application working path.

Please refer to the according sample projects for the detailed information.

7FAQ

7.1What's the serial communication configure parameters?

On the hardware version of 4.0.0 or new, access the DIO using the PC local bus, but the light control use serial port 3 on the NP series products, the default Baudrate is 115200bps.

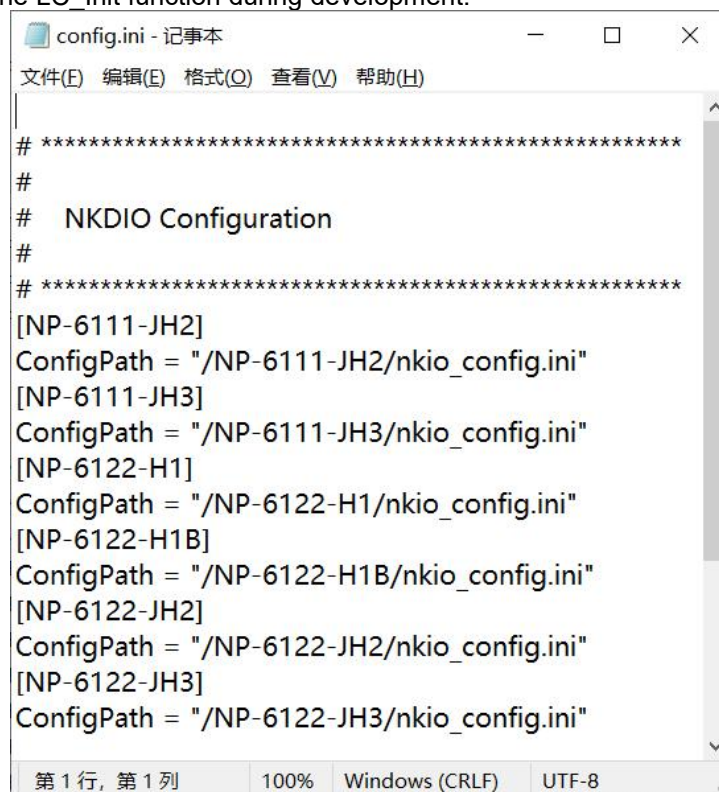
7.2How to check the hardware version?

You can use the test tool connect to the controller, the version information will be upload and display in the page after connected.

Please remind that the functions in this document only can be used on the hardware version 4.0.0 or newer.

7.3How to get the configure profile for different IO board?

The configuration of the IO board is illulstrated in the ini file. In the path of the SDK installation, the path of configure profile is listed in the config.ini file, you must import the according Light Control configure profile in the LC_Init function during development.



```
config.ini - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

# *****
#
#   NKDIO Configuration
#
# *****
[NP-6111-JH2]
ConfigPath = "/NP-6111-JH2/nkio_config.ini"
[NP-6111-JH3]
ConfigPath = "/NP-6111-JH3/nkio_config.ini"
[NP-6122-H1]
ConfigPath = "/NP-6122-H1/nkio_config.ini"
[NP-6122-H1B]
ConfigPath = "/NP-6122-H1B/nkio_config.ini"
[NP-6122-JH2]
ConfigPath = "/NP-6122-JH2/nkio_config.ini"
[NP-6122-JH3]
ConfigPath = "/NP-6122-JH3/nkio_config.ini"

第 1 行, 第 1 列    100%    Windows (CRLF)    UTF-8
```